WATI Reservation Setup Guide

Architecture, dashboard layout, data model, and rollout plan for WhatsApp table bookings

Prepared for	CleverBiz AI restaurant dashboard
Date	31 May 2026
Recommended approach	WATI as customer channel; CleverBiz backend as reservation source of truth

Executive recommendation

Use WATI for customer conversation, collection of booking details, confirmations, reminders, and support handoff. Keep the CleverBiz backend as the only source of truth for availability, reservation status, table assignment, conflict control, and dashboard display.

1. Operating Model

The cleanest setup is channel-first, not WATI-first. WATI should collect structured booking data and communicate with the customer. The CleverBiz backend should decide whether a booking can be accepted and should own the reservation record.

Layer	Responsibility	Implementation notes
WhatsApp + WATI	Customer conversation, chatbot questions, templates, quick replies, and message statuses.	Use keyword triggers and a structured bot flow. Send final booking data to the backend through a webhook node.
Integration API	Receives WATI payloads, validates source, handles duplicates, and normalizes fields.	Return 200 OK quickly. Process idempotently because WATI retries failed webhooks.
Reservation engine	Availability, capacity, table allocation, staff approval rules, and status transitions.	This layer prevents overbooking and keeps future Google/phone/manual bookings consistent.
Restaurant dashboard	Operational view for host/manager/staff.	Show source, status, conflict warnings, customer details, and direct staff actions.
Outbound WATI API	Confirmation, rejection, reminders, modifications, and cancellation messages.	Use session messages when inside the active WhatsApp window; use approved templates when outside it.

2. Customer Booking Flow

1. Customer sends a booking keyword such as book, reserve, reservation, table, hi, or hello.
2. WATI bot asks structured questions and stores each answer in variables or custom attributes.
3. Bot shows a summary and asks the customer to confirm before creating the booking request.
4. WATI calls the CleverBiz webhook with the collected fields and conversation metadata.
5. Backend validates the payload, checks availability, applies auto-confirm rules, and creates the reservation.
6. Dashboard updates through the normal reservations API and websocket/event flow.
7. Customer receives a WATI confirmation, pending-approval message, rejection, or alternative time options.

Recommended WATI Bot Fields

Field	Input type	Required	Dashboard display
Name	Question box	Yes	Customer name in reservation list and drawer
Phone	WhatsApp number plus confirmation	Yes	Customer phone, WhatsApp action link
Date	Date text or list	Yes	Reservation date filter and timeline placement
Time	Time text or list	Yes	Reservation card, timeline slot
Guests	Number	Yes	Party size, capacity check, KPI totals
Seating preference	List: indoor, outdoor, smoking, window, private, any	Recommend ed	Preference badge and host note
Occasion	List or free text	Optional	Occasion chip: birthday, anniversary, business, etc.
Special requests	Free text	Optional	Highlighted in drawer and list warning icon
Marketing consent	Button: yes/no	Optional	CRM consent flag if loyalty/marketing is enabled

Availability Response Logic

Condition	Backend action	Customer response
Available small party	Create reservation as confirmed.	Send confirmed booking message immediately.
Available but needs review	Create reservation as pending_staff_confirmation or needs_approval.	Tell customer the request is received and staff will confirm shortly.
Slot full	Do not confirm. Return nearest valid slots.	Offer alternative times through WATI quick replies or list message.
Restaurant closed	Reject request with next available opening window.	Send polite closed-hours message with alternative dates.
Duplicate webhook retry	Return existing reservation by idempotency key.	No duplicate customer message unless status changed.

3. Backend Data Structure

The reservation record should be source-aware from day one. This avoids rework when Google reservations, manual phone bookings, walk-ins, and future integrations are added.

Field group	Fields	Purpose
Identity	id, restaurantId, customerName, customerPhone, customerEmail	Connects the reservation to restaurant, customer, CRM, and WATI contact.
Booking	partySize, reservationDate, reservationTime, durationMinutes, tableId, tableName	Drives timeline, table allocation, and capacity checks.
Preferences	areaPreference, occasion, specialRequests	Gives host/staff enough context before guest arrival.
Workflow	status, assignedStaffId, createdAt, updatedAt	Controls operational state and auditability.
Source	source, sourceChannel, externalConversationId, externalMessageId, idempotencyKey, rawPayload	Links the booking back to WATI and prevents duplicate webhook-created reservations.

Reservation Statuses

Status	Meaning	Badge color
pending	Request received but not confirmed yet.	Amber
needs_approval	Requires staff decision due to party size, special request, or capacity risk.	Purple
confirmed	Booking is accepted and customer has been notified.	Blue
seated	Guest has arrived and is seated.	Green
completed	Visit finished.	Slate
cancelled	Booking cancelled by customer or staff.	Red
rejected	Restaurant declined the request.	Red
no_show	Guest did not arrive.	Rose

Required API Surface

Endpoint	Purpose
POST /api/integrations/wati/reservations/webhook	Main ingestion endpoint from WATI chatbot webhook node.
POST /api/integrations/wati/events	Receives message status callbacks: sent, delivered, read, failed.
GET /api/reservations	Dashboard list with filters for date, status, source, and search.
GET /api/reservations/:id	Full reservation detail with event timeline and WATI metadata.
PATCH /api/reservations/:id	Edit customer details, time, table, notes, and status.
POST /api/reservations/:id/confirm	Confirm booking and send WATI confirmation.
POST /api/reservations/:id/reject	Reject booking and send WATI rejection or alternative time message.
POST /api/reservations/:id/send-whatsapp	Send manual WhatsApp update from the dashboard.

4. Restaurant Dashboard Layout

Dashboard principle

The reservations page should behave like an operations console. Staff need to know who is coming, when they are coming, what action is needed, and whether the customer has been notified.

Page Header

Element	Content
Title	Reservations
Subtitle	Today, date, and live sync status
Primary actions	New Manual Booking, Sync WATI, Settings
Status indicator	Live WATI webhook / Last event received timestamp

KPI Cards

Card	What it shows
Today's Reservations	Total reservations scheduled for the selected date.
Confirmed	Bookings accepted and customer notified.
Pending Approval	Requests waiting for staff decision.
Seated Now	Active seated reservations.
No Shows	Bookings marked no-show for the selected period.
WhatsApp Requests	Bookings created from WATI.

Main Views

View	Best use	Layout
Timeline view	Default host/manager view for today.	Time slots down the page with reservation cards placed by time.
Board view	Operational workflow.	Columns: Pending, Confirmed, Seated, Completed, Cancelled.
Table view	Search, admin review, and historical records.	Sortable table with filters, source badges, and bulk actions.

Reservation Card

- Time and duration, shown first for fast scanning.
- Customer name and phone number.
- Party size and table assignment.
- Status badge and source badge: WATI, Manual, Google future, Walk-in.
- Area preference and occasion chips.
- Special request indicator if notes exist.
- Last WhatsApp message state: sent, delivered, read, or failed.

Detail Drawer

Section	Fields and controls
Customer	Name, phone, WhatsApp profile/source, previous visits, CRM link.
Booking	Date, time, guests, duration, table, area, occasion, special requests.
WhatsApp	Conversation ID, message status, open in WATI, send confirmation, send reminder, send cancellation.
Timeline	Request received, availability checked, confirmed/rejected, message sent, message read.
Actions	Confirm, reject, edit, assign table, mark seated, complete, no-show, cancel.

5. Staff Actions

Action	When used	System behavior
Confirm	Pending or needs-approval request can be accepted.	Update status to confirmed, reserve capacity/table, send WATI confirmation.
Reject	Slot unavailable or restaurant declines request.	Update status to rejected, send WATI rejection or alternatives.
Suggest new time	Requested slot unavailable but alternatives exist.	Send WATI quick reply/list with valid slots.
Edit booking	Customer asks to modify date, time, guests, or table.	Re-run availability check before saving.
Assign table	Host wants table-level control.	Set tableId/tableName and detect conflicts.
Mark seated	Guest arrives.	Set status seated and optionally create table occupancy.
No-show	Guest does not arrive.	Set status no_show and optionally notify customer.
Cancel	Customer or staff cancels.	Release capacity/table and send cancellation message.

6. WATI Settings

Add a WATI settings area inside the restaurant dashboard so managers can configure the integration without code changes.

Setting	Purpose
Enable WhatsApp reservations	Master switch for WATI reservation ingestion.
WATI API endpoint	Tenant-specific WATI base URL.
Bearer token	Secret used for outbound WATI API calls. Store encrypted/server-side only.
Webhook verification token	Validates inbound WATI events where supported.
Default duration	Default reservation block, for example 90 minutes.
Buffer time	Cleaning/turnover gap between bookings.
Auto-confirm rules	Party size, time window, capacity threshold, same-day policy.

Templates	reservation_confirmed, reservation_pending, reservation_cancelled, reservation_reminder.
Opening hours and closed days	Used by availability engine and WATI alternative-time suggestions.

7. Reliability, Security, and Edge Cases

- Always respond 200 OK to accepted webhook payloads and process duplicates idempotently.
- Use an idempotency key built from restaurantId, WATI conversation ID, message ID, and booking summary hash.
- Store rawPayload for debugging, but do not expose raw secrets or tokens in the dashboard.
- Never auto-confirm if capacity or table allocation is uncertain.
- Use approved WATI templates for messages outside the active WhatsApp session window.
- Log every status change in a reservation event table for auditability.
- Use timezone-aware parsing. Store UTC in the backend and display restaurant local time in the dashboard.
- Normalize phone numbers into E.164 format for customer matching and CRM linkage.

8. Rollout Plan

Phase	Scope	Done when
1. Schema and source-aware UI	Add WATI/source/status fields, badges, filters, drawer sections.	Manual and WATI-looking records display correctly.
2. WATI ingestion	Webhook endpoint, payload normalization, idempotency, reservation creation.	WATI bot can create pending reservations with no duplicates.
3. Availability engine	Opening hours, slot capacity, party size rules, conflict detection.	Unavailable slots are rejected or return alternatives.
4. Staff actions	Confirm, reject, edit, assign table, seated, no-show, cancel.	Actions update dashboard and send correct WATI messages.
5. Automation	Reminders, status callbacks, failed-message warnings.	Customers receive reminders and staff see delivery/read status.
6. Advanced operations	Timeline/board/table views, table-level allocation, CRM linkage.	Host can run the floor from the reservations page.

9. Acceptance Checklist

- A customer can complete the WATI booking flow without staff assistance.
- A WATI webhook creates exactly one reservation even if WATI retries the event.
- Dashboard shows WATI source badge, customer phone, party size, time, status, preferences, and special request.
- Staff can confirm or reject a booking from the drawer.
- Customer receives the correct WhatsApp message after staff action.
- Unavailable times are not auto-confirmed.
- Dashboard works on desktop, tablet, and mobile with drawer/action-sheet behavior.
- All booking events are visible in the reservation timeline.

10. Sources

WATI API overview: https://support.wati.io/en/articles/11462487-wati-apis-overview

WATI webhook setup and retry behavior: https://support.wati.io/en/articles/14111740-how-to-set-up-and-use-webhooks-in-wati

WATI chatbot webhook node: https://support.wati.io/en/articles/11463025-advance-chatbot-builder-using-webhook-node